

声明

为开微电子保有在不事先通知而修改这份文档利。为开微电子认为提供的信息是准确可信的，尽管这样，难免也有疏漏之处。我们一直在不断的改进我们的产品及说明文档的品质。我们努力保证这份文档的说明是准确的，但也有可能存在一些我们未曾发现的失误。如果您发现了文档中的任何疑问或者错失的地方，请及时联系我们。您的理解和支持将使得这份文档更加完善。

版本	发表日期	备注
V1.0	2017.11	创建文件
V2.0	2019.3	更新

WK2114 应用笔记



1. 产品概述

WK2114 是 UART 接口的 4 通道 UART 器件，WK2114 将一个标准 3 线异步串口 (UART) 扩展成为 4 个增强功能串口 (UART)。

扩展的子通道的UART具备如下功能特点：

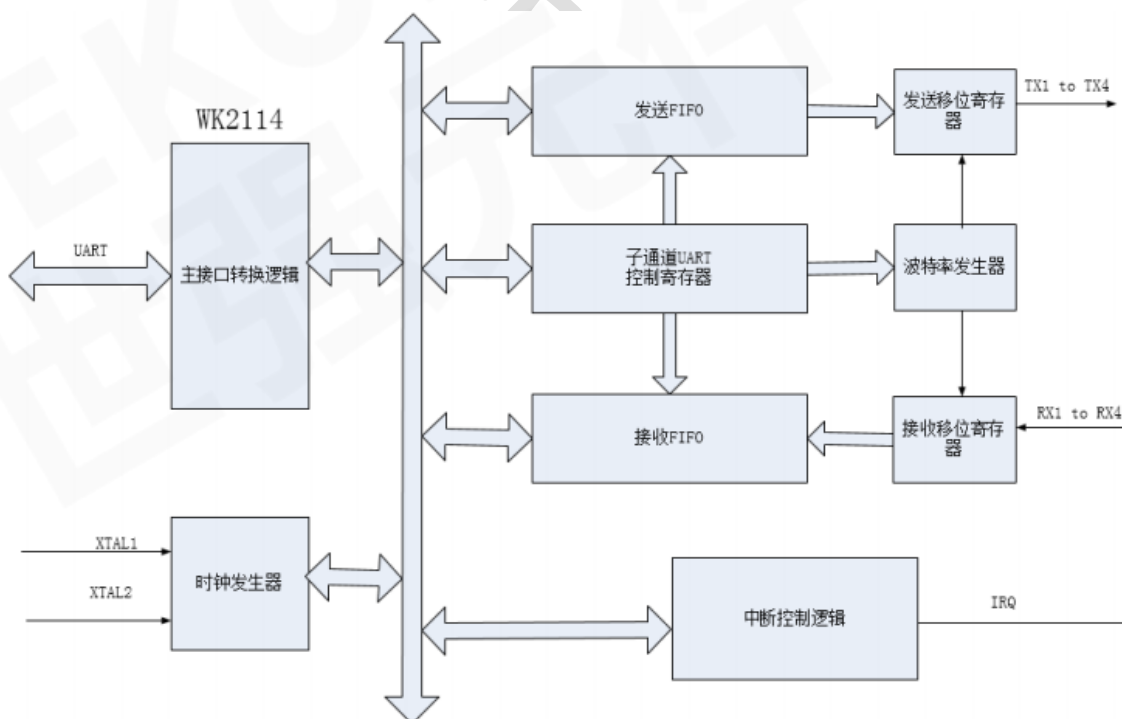
- 每个子通道UART的波特率、字长、校验格式可以独立设置，最高可以提供2Mbps的通信速率。
- 每个子通道可以独立设置工作在IrDA红外通信。
- 每个子通道具备收/发独立的256 级FIFO，FIFO的中断可按用户需求进行编程触发点且具备超时中断功能。

WK2114 采用 SOP20L 绿色环保无铅封装，可以工作在 2.5~5.0V 的宽工作电压范围，具备可配置自动休眠/唤醒功能。

[注]：SPI™ 为MOTOROLA公司的注册商标。

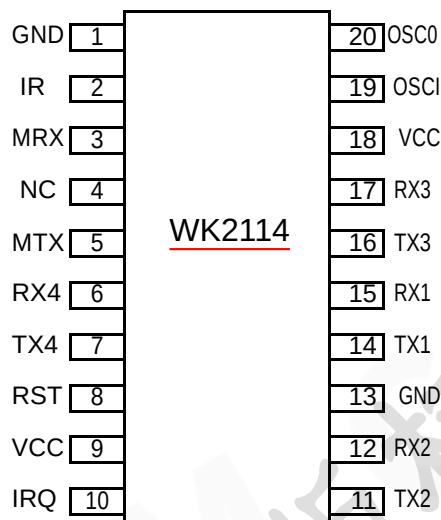
2 . 基本特性

2.1 原理框图

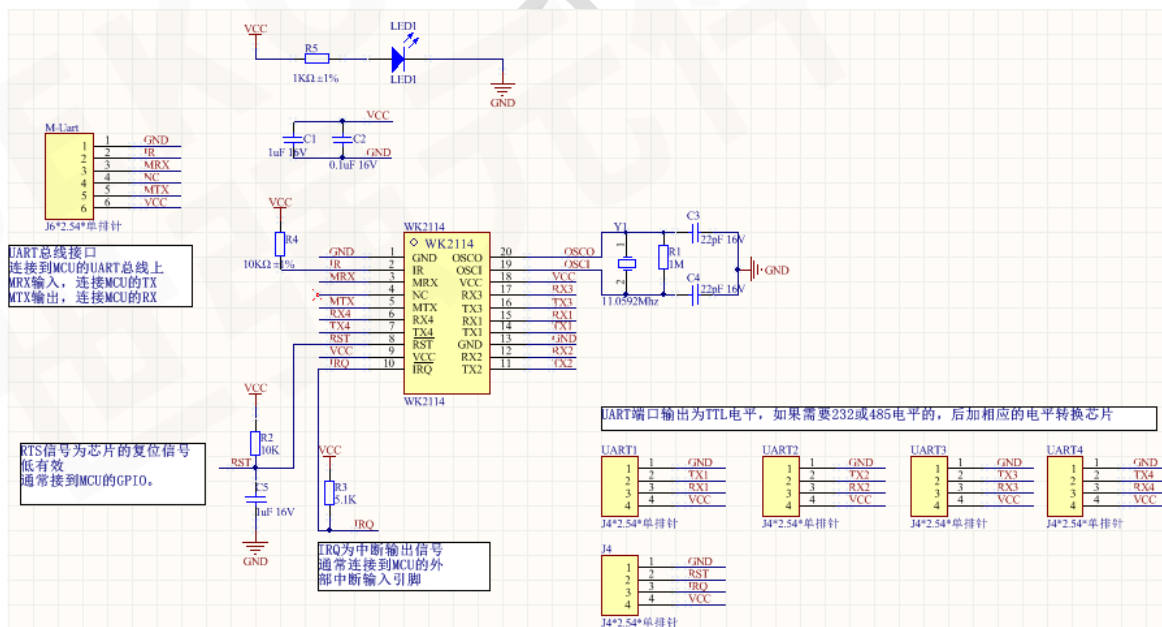


2.2 封装引脚图

WK2114 封装引脚图 SOP20L:



2.3 评估板硬件原理图



1、从原理图上分析，WK2114 和 MCU 之间的链接主要有 3 部分：

A、UART 总线：WK2114 通过 UART 和 MCU 进行数据通信。UART 会传输命令字节和数据

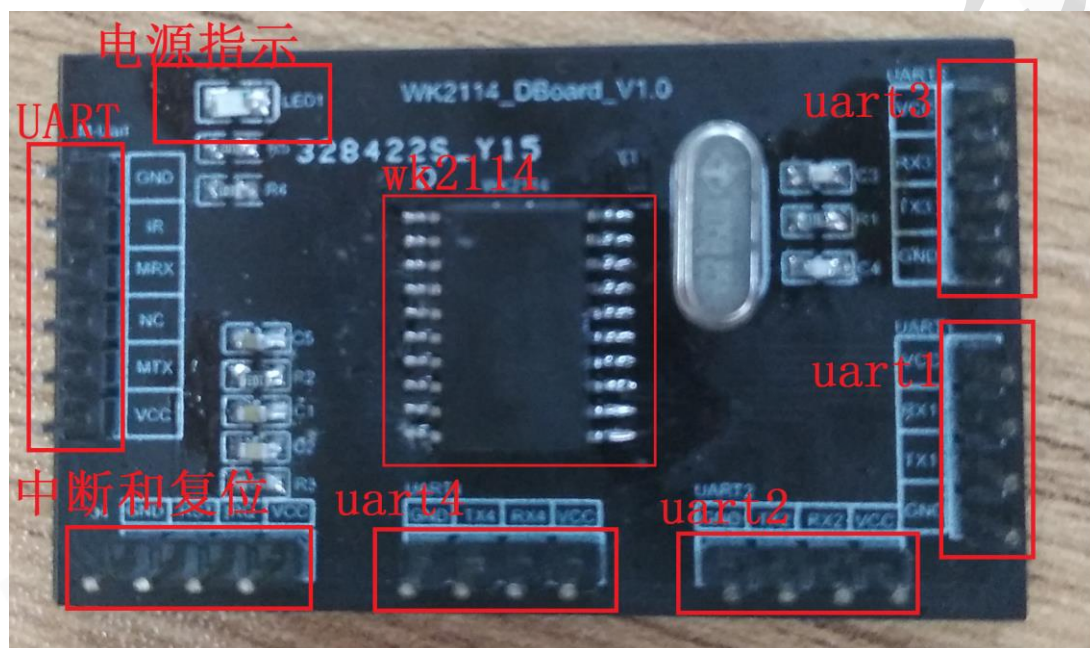
字节。

B、中断：WK2114 的 IRQ 连接到 MCU 的外部中断输入引脚。这个连接不是必须的，如果没有连接到 MCU 的外部中断输入引脚，那么就只能采用查询的编程方式实现对子串口的数据收发，效率较低。

C、复位控制：WK2114 的复位引脚，低电平有效，时间长度为 10ms。复位以后，WK2114 的所有寄存器值恢复到默认值，UART 总线上的命令解析也会同时复位。

2、时钟电路：通常采用无源晶振，晶振大小和子串口波特率相关，具体关系看数据手册波特率编程章节；和晶振并联的匹配电阻是 1M 欧的，是必须的。

2.4 硬件评估板硬件介绍



- 1、WK2114 可以实现主串口扩展 4 串口。
- 2、中断和复位，中断引脚需要连接到 MCU/CPU 的外部中断引脚;复位引脚 RST 可以连接到 GPIO,特别是当主接口是主 UART 时。
- 3、子串口 1~子串口 4，每个子串口都独立的，可以同时收发数据
- 4、电源指示灯，指示评估板是否通电。

3. UART 接口时序与软件编程

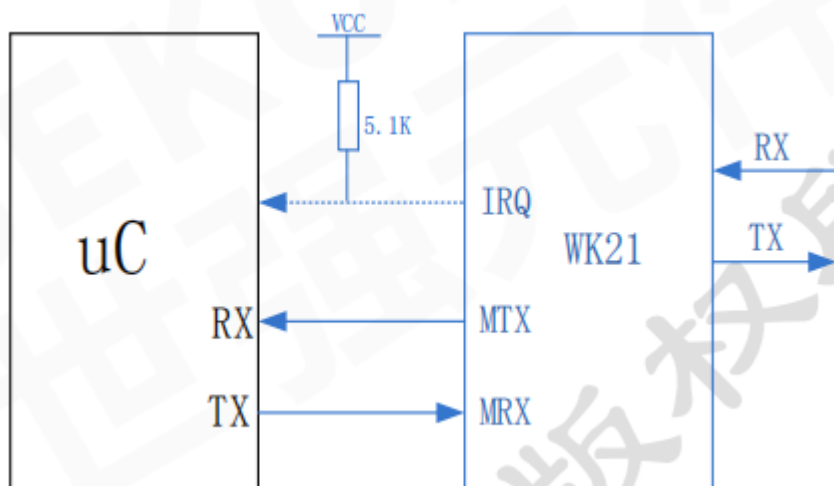
3.1 WK2114 UART 主接口操作时序综述

当 WK2114 的主接口为 UART 时，仅需要 RX，TX 连接主机。采用标准的 UART 协议进行通信。主接口

UART 可以实现波特率自适应。上电复位后，先向 WK2114 写入 0x55，WK2114 可以根据写入的数据自动

测得此时控制器的波特率并把主接口 UART 的波特率锁定，以后就以此波特率进行通信；如果主接口需要再次更换波特率，需要对芯片硬件复位，然后再次进行波特率测试和锁定。

WK2114 与主机的连接如下



3.2 WK2114 UART波特率自适应时序操作

主接口UART可以实现波特率自适应。WK2114硬件复位后，先向WK2114写入0x55，WK2114可以根据写入的数据自动测得此时MCU的波特率并把主接口UART的波特率锁定，以后就以此波特率进行通信；如果主接口需要再次更换波特率，需要对芯片硬件复位，然后再次进行波特率测试和锁定。

如下为demo程序中波特率自适应的一段程序。

```
void Wk_BaudAdaptive(void)
{
    /*初始化拉高RST引脚10ms*/
    GPIO_SetBits(GPIOA, GPIO_Pin_4);
    delay_ms(10);
    /*拉低RST引脚10ms, 复位WK芯片*/
    GPIO_ResetBits(GPIOA, GPIO_Pin_4); /*拉低*/
    delay_ms(10);
    /*拉高RST引脚, 完成复位, wk进入正常工作状态*/
    GPIO_SetBits(GPIOA, GPIO_Pin_4);
    delay_ms(20);
    /*复位完成以后, 发送0x55, 实现主串口波特率匹配*/
    uart_sendByte(0x55);
    delay_ms(100);
}
```

3.3 WK2114 UART写寄存器时序分析

写操作时, 先向WK2114的MRX写入一个命令字节 (Command Byte), 随后写入相应的数据字节, 其操作时序 (无校验模式) 如图所示:



UART	控制字节 CMD								1 个数据字节 DB(下行)							
BIT	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
TX	0	0	C1	C0	A3	A2	A1	A0	D7	D6	D5	D4	D3	D2	D1	D0
RX																

写全局寄存器编程参考:


```

22 /*****WkWriteGReg*****/
23 //函数功能：写全局寄存器函数（前提是该寄存器可写，
24 //某些寄存器如果你写1，可能会自动置1，具体见数据手册）
25 //参数：
26 //     greg:为全局寄存器的地址
27 //     dat:为写入寄存器的数据
28 /*****/
29 void WkWriteGReg(unsigned char greg,unsigned char dat)
30 {   u8 cmd;
31     cmd=0|greg;
32     uart_sendByte(cmd); //写指令，对于指令的构成见数据手册
33     uart_sendByte(dat); //写数据
34 }

```

注意：全局寄存器是 6bit 的地址，所以对应 CMD 字节中的 C1,C0,A3,A2,A1,A0

写子串口寄存器编程参考：

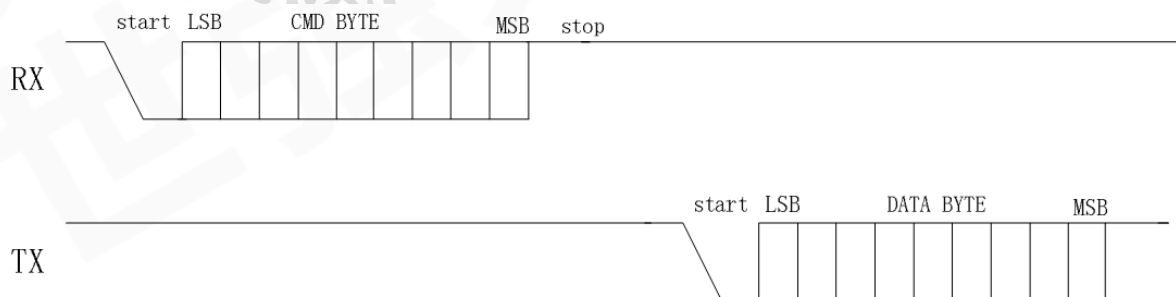
```

49 /*****WkWriteSReg*****/
50 //函数功能：
51 //参数：port:为子串口
52 //     sreg:为子串口寄存器
53 //     dat:为写入寄存器的数据
54 //注意：在子串口被打通的情况下，向FDAT写入的数据会通过TX引脚输出
55 /*****/
56 void WkWriteSReg(u8 port,u8 sreg,u8 dat)
57 {   u8 cmd;
58     cmd=0x0|((port-1)<<4)|sreg;
59     uart_sendByte(cmd); //写指令，对于指令的构成见数据手册
60     uart_sendByte(dat); //写数据
61 }

```

3.4 WK2114 UART读寄存器时序分析

读操作时，先向WK2114的RX写入命令字节，相应的数据字节从MTX读取，其操作时序（无校验模式）如图10.2.2所示



读全局寄存器编程参考：

```

35 /*****WkReadGReg*****/
36 //函数功能: 读全局寄存器
37 //参数:
38 //     greg: 为全局寄存器的地址
39 //     rec: 返回的寄存器值
40 /*****/
41 u8 WkReadGReg(unsigned char greg)
42 {
43     u8 cmd, rec;
44     cmd=0x40|greg;
45     uart_sendByte(cmd); //写指令, 对于指令的构成见数据手册
46     rec=uart_recByte(); //接收返回的寄存器值
47     return rec;
48 }

```

读子串口寄存器编程参考：

```

63 /*****WkReadSReg*****/
64 //函数功能: 读子串口寄存器
65 //参数: port为子串口端口号
66 //     sreg: 为子串口寄存器地址
67 //     rec: 返回的寄存器值
68 /*****/
69 u8 WkReadSReg(u8 port, u8 sreg)
70 {
71     u8 cmd, rec;
72     cmd=0x40|((port-1)<<4)|sreg;
73     uart_sendByte(cmd); //写指令, 对于指令的构成见数据手册
74     rec=uart_recByte(); //接收返回的寄存器值
75     return rec;
76 }

```

3.5 WK2114 UART写FIFO时序分析

UART	控制字节 CMD								[N3 N2 N1 N0] 个数据字节 DB(下行)							
BIT	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
TX	1	0	C1	C0	N3	N2	N1	N0	D7	D6	D5	D4	D3	D2	D1	D0
RX																

注意：写FIFO操作，就是通过该指令直接从子串口的FIFO写入数据，单次数据个数不超过16个Byte。

编程参考：

```

76 /*****WkWriteSFifo*****/
77 //函数功能: 向子串口fifo写入需要发送的数据
78 //参数: port: 为子串口
79 //     *dat: 写入数据
80 //     num: 为写入数据的个数, 单次不超过16个字节
81 //注意: 通过该方式写入的数据, 被直接写入子串口的缓存FIFO, 然后被发送
82 /*****/
83 void WkWriteSFifo(u8 port, u8 *dat, u8 num)
84 {
85     u8 cmd, i;
86     cmd=0x80|((port-1)<<4)|(num-1);
87     uart_sendByte(cmd); //写指令, 对于指令构成见数据手册
88     for(i=0; i<num; i++)
89     {
90         uart_sendByte(*dat+i); //写数据
91     }
92 }

```


3.6 WK2114 UART读FIFO时序分析

UART	控制字节 CMD								[N3 N2 N1 N0]个数据字节 DB(上行)							
BIT	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
TX	1	1	C1	C0	N3	N2	N1	N0								
RX									D7	D6	D5	D4	D3	D2	D1	D0

注意：写FIFO操作，就是通过该指令直接从子串口的FIFO读出数据，单次数据个数不超过16个Byte。

编程参考：

```

94 /*****WkReadSFifo*****/
95 //函数功能:从子串口的fifo中读出接收到的数据
96 //参数: port:为子串口
97 //      *rec: 接收到的数据
98 //      num: 读出的数据个数。
99 //注意: 通过该方式读出子串口缓存中的数据。单次不能超过16个。
100 /*****/
101 void WkReadSFifo(u8 port,u8 *rec,u8 num)
102 {
103     u8 n,cmd;
104     cmd=0xc0|((port-1)<<4)|(num-1);
105     uart_sendByte(cmd);
106     for(n=0;n<num;n++)
107     {
108         rec[n]=uart_recByte();
109     }
110 }

```

4. WK2114 寄存器介绍

4.1 WK2114 寄存器综述

WK2114 串口扩展芯片的寄存器可以分为 3 类：全局寄存器、页控制寄存器、子串口寄存器。

- 1、全局寄存器：主要是控制主接口或者是某个子串口功能的总开关
- 2、页控制寄存器：由于子串口寄存器分布在不同的页上，如果要写不同页上的子串口寄存器，需要通过该寄存器来切换页。芯片复位后，子串口的页指针是默认到 PAGE0 的，如果需要写 PAGE1 上的寄存器，那么就通过该寄存器来切换。
- 3、子串口寄存器：完成子串口功能设置和状态指示。每个子串口都有一组独立的子串口寄存器，但是每个寄存器都是一样的，我们通过子串口的端口加也区分。

4.2 WK2114 全局寄存器综述

全局寄存器：主要是控制主接口或者是某个子串口功能的总开关。

寄存器地址[5:0]	寄存器名称	类型	寄存器功能描述
000000	GENA	R/W	全局控制寄存器

000001	GRST	R/W	全局子串口复位寄存器
000010	GMUT	R/W	全局主串口控制寄存器
010000	GIER	R/W	全局中断寄存器
010001	GIFR	R	全局中断标志寄存器

GENA：全局控制寄存器，主要是控制子串口时钟。只有在使能子串口时钟使能的情况下，才能操作子串口相关寄存器。

GRST：全局子串口复位寄存器：该寄存器能从软件上单独复位某个子串口，向该寄存器写入1，完成复位以后，自动置0。

GIER：全局中断寄存器，该寄存器控制每个子串口的总的中断。(如果需要使用子串口的中断功能，通常需要先使能 GIER,然后使能 SIER，子串口的中断才能生效)

GIFR：全局中断标志寄存器，该寄存器指示某个子串口是否有中断，需要知道具体的中断情况，需要查询子串口的 SIFR 寄存器确定。

4.3 WK2114 子串口寄存器介绍

每个子串口都有单独的一组子串口寄存器

表7.2 子串口控制寄存器

寄存器地址 [3:0]	寄存器名称	类型	寄存器功能描述	
(C1,C0) 0011	SPAGE	R/W	子串口页控制寄存器	
(C1,C0) 0100	SCR	R/W	子串口控制寄存器	SPAGE0
(C1,C0) 0101	LCR	R/W	子串口配置寄存器	SPAGE0
(C1,C0) 0110	FCR	R/W	子串口 FIFO 控制寄存器	SPAGE0
(C1,C0) 0111	SIER	R/W	子串口中断使能寄存器	SPAGE0
(C1,C0) 1000	SIFR	R/W	子串口中断标志寄存器	SPAGE0
(C1,C0) 1001	TFCNT	R	子串口发送 FIFO 计数寄存器	SPAGE0

(C1,C0) 1010	RFCNT	R	子串口接收 FIFO 计数寄存器	SPAGE0
(C1,C0) 1011	FSR	R	子串口 FIFO 状态寄存器	SPAGE0
(C1,C0) 1100	LSR	R	子串口接收状态寄存器	SPAGE0
(C1,C0) 1101	FDAT	R/W	子串口 FIFO 数据寄存器	SPAGE0
(C1,C0) 0100	BAUD1	R/W	子串口波特率配置寄存器高字节	SPAGE1
(C1,C0) 0101	BAUD0	R/W	子串口波特率配置寄存器低字节	SPAGE1
(C1,C0) 0110	PRES	R/W	子串口波特率配置寄存器小数部分	SPAGE1
(C1,C0) 0111	RFTL	R/W	子串口接收 FIFO 中断触发点配置寄存器	SPAGE1
(C1,C0) 1000	TFTL	R/W	子串口发送 FIFO 中断触发点配置寄存器	SPAGE1

(C1, C0) : 子通道号, 00 ~ 11分别对应子串口1到子串口4

4.4 发送数据过程

发送数据过程：发送数据就是 CPU 通过主接口把需要发送到子串口的数据发送到 FIFO，然后 FIFO 中的数据在按照相应的波特率发送出去。

当我们需要把数据发送给和 [WK2114](#) 连接的子设备，那么我们就需要通过主接口向 [WK2114](#) 写入相应的数据，然后数据在经过 [WK2114](#) 传输给指定的子串口（子设备）。那么这就完成了一次数据的发送。具体发送过程可以参考实例说明。

4.5 接收数据过程

数据接收过程是这样的，当在子串口的接收功能使能的情况下，子串口 RX 引脚接收到的数据暂存在子串口的接收 FIFO 当中，需要 CPU 主动去 [WK2114](#) 的接收 FIFO 当中取。具体接收过程可以参考实例说明。

4.6 中断结构分析

中断对于 [WK2114](#) 的意义：对于 [WK2114](#) 的操作，都是 MCU 通过总线去读写 [WK2114](#) 的寄存器，这些对于 [WK2114](#) 来说都是被动的操作，被动的接受 MCU 过来的数据，当然有时候

我们也有些消息需要及时的通知 MCU，那么中断输出信号就是为实现这项功能而设立的。

IRQ 中断信号输出引脚通常会接到 MCU 的外部中断输入引脚。

WK2114 中断分为两级，一级是总中断开关。二级是子串口中断。每个子串口都有自己的独立的中断系统。

中断的使用方式：

- A. 首先使能总中断开关
- B. 使能相应子串口的相应中断
- C. 对于接收和发送 FIFO 触点中断需要设置中断触点，也就是中断产生的条件中断处理方式：

当中断来了以后我们应该怎么判断全局中断

- D. 首先判断是哪个子串口的中断，读取全局中断标志寄存器
- E. 判断具体的中断源，读取子串口中断标志寄存器

4.7 调试技巧及分析

调试技巧：

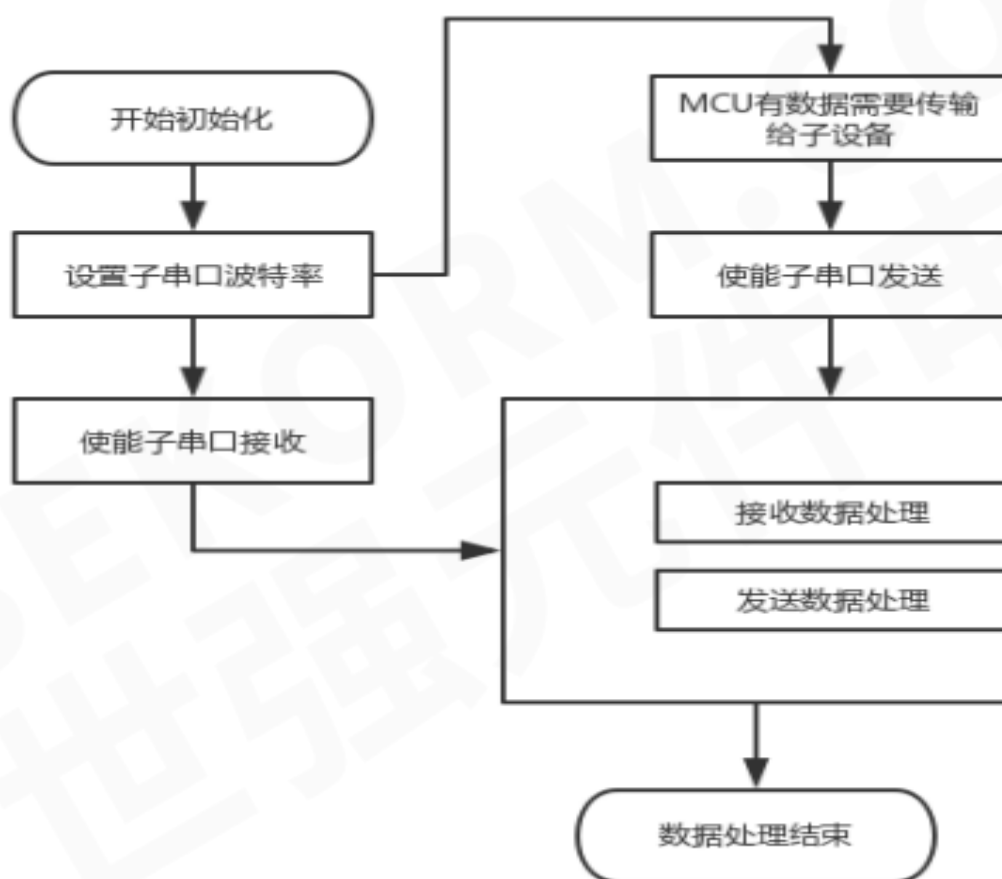
1. 电路硬件检查：
 - A. 首先检查电源，看芯片电源正和 GND 是否正常（建议电源正和地和 MCU 相同）。芯片焊接是否有虚焊，过焊，和短路的情况。
 - B. 查看晶振是否起振，起振是否正常；然后检查复位引脚，复位后应该保持高电平。
2. 软件调试：在保证硬件没有问题的情况下可进行软件调试
 - A. 上电后最好进行复位。保证芯片处于一个初始状态（在调试阶段复位很关键，很多调试不正常可能都是由于前期错误操作导致的，只要复位，可避免前期错误操作带来的影响）
 - B. 通常我们调试软件都是先调试主接口通信。WK2114 是工作在主接口为标准的三线 UART 串口（RX, TX, GND），无需其它地址信号、控制信号线，仅需要 RX, TX 连接主机。采用标准的 UART 协议进行通信。主接口可以实现波特率自适应。上电复位后，先向 WK2114 写入 0x55，WK2114 可以根据写入的数据自动测得此时控制器的波特率并把主接口 UART 的波特率锁定，以后就以此波特率进行通信；如果主接口需要再次更换波特率，需要对芯片硬件复位，然后再次进行波特率测试和锁定。
 - C. 在时序满足情况下，我们可以通过读 WK2114 芯片某些值比较固定的寄存器，来判断主接口是否通信成功。比如：GENA，在没有使能的情况下，默认 GENA 的值为 0xF0。

- D. 读调试成功好，然后再调试写。可以写一些相关寄存器的值（GENA），通过读来验证写寄存器是否成功。
- E. 配置发送数据和接收数据相关寄存器。主要是使能子串口时钟和使能发送和接收，配置子串口相关波特率等。（具体操作可以参考下面编程实例）。在调试过程中可以先将配置的寄存器读出来，看一下是否配置成功。
- F. 如果需要用到中断，WK2114IRQ 为中断输出信号引脚，低电平有效。
- G. 最后，你可以正常进行收发数据了。

5. 编程实例

5.1 WK2114 程序结构分析

程序主体流程图



5.2 WK2114 程序实例

WK2114.c

复位引脚初始化

```
/******WK_RstInit******/
```

//函数功能:wk 芯片需要用 MCU 的 GPIO 去控制 RST 引脚, 本函数通过 stm32 的 A4 引脚连接 WK 的 RST 引脚

//初始化 STM32 的 A4 引脚。

//

//*****/

void WK_RstInit(void)

{

GPIO_InitTypeDef GPIO_InitStructure;

RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE); //使能 PA,PD 端口时钟

GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4; //PA.4 端口配置

GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; //推挽输出

GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz; //IO 口速度为 50MHz

GPIO_Init(GPIOA, &GPIO_InitStructure); //根据设定参数初始化 GPIOA.4

GPIO_SetBits(GPIOA,GPIO_Pin_4); //PA.4 输出高

}

读, 写寄存器函数, 分全局寄存器和子串口寄存器

//*****WkWriteGReg*****/

//函数功能: 写全局寄存器函数 (前提是该寄存器可写, 某些寄存器如果你写 1, 可能会自动置 1, 具体见数据手册)

//参数:

// greg: 为全局寄存器的地址

// dat: 为写入寄存器的数据

//*****/

void WkWriteGReg(unsigned char greg, unsigned char dat)

{

u8 cmd;

cmd=0|greg;

uart_sendByte(cmd); //写指令, 对于指令的构成见数据手册

uart_sendByte(dat); //写数据

}

//*****WkReadGReg*****/

//函数功能: 读全局寄存器

//参数:

// greg: 为全局寄存器的地址

// rec: 返回的寄存器值

//*****/

u8 WkReadGReg(unsigned char greg)

{

u8 cmd, rec;

cmd=0x40|greg;

uart_sendByte(cmd); //写指令, 对于指令的构成见数据手册


```
    rec=uart_recByte();    //接收返回的寄存器值
    return rec;
}

/*****WkWriteSReg*****/
//函数功能:
//参数 : port:为子串口
//      sreg:为子串口寄存器
//      dat:为写入寄存器的数据
//注意 : 在子串口被打通的情况下, 向 FDAT 写入的数据会通过 TX 引脚输出
/*****/
void WkWriteSReg(u8 port,u8 sreg,u8 dat)
{
    u8 cmd;
    cmd=0x0|((port-1)<<4)|sreg;
    uart_sendByte(cmd); //写指令, 对于指令的构成见数据手册
    uart_sendByte(dat);//写数据
}

/*****WkReadSReg*****/
//函数功能: 读子串口寄存器
//参数 : port 为子串口端口号
//      sreg:为子串口寄存器地址
//      rec:返回的寄存器值
/*****/
u8 WkReadSReg(u8 port,u8 sreg)
{
    u8 cmd,rec;
    cmd=0x40|((port-1)<<4)|sreg;
    uart_sendByte(cmd); //写指令, 对于指令的构成见数据手册
    rec=uart_recByte();    //接收返回的寄存器值
    return rec;
}
```

读写FIFO函数

```
/*****WkWriteSFifo*****/
//函数功能:向子串口 fifo 写入需要发送的数据
//参数 : port:为子串口
//      *dat:写入数据
//      num : 为写入数据的个数, 单次不超过 16 个字节
//注意 : 通过该方式写入的数据, 被直接写入子串口的缓存 FIFO, 然后被发送
/*****/
```

```
void WkWriteSFifo(u8 port,u8 *dat,u8 num)
{
    u8 cmd,i;
    cmd=0x80|((port-1)<<4)|(num-1);
    uart_sendByte(cmd); //写指令,对于指令构成见数据手册
    for(i=0;i<num;i++)
    {
        uart_sendByte( *(dat+i)); //写数据
    }
}

/*****WkReadSFifo*****/
//函数功能:从子串口的 fifo 中读出接收到的数据
//参数 : port:为子串口
//      *rec : 接收到的数据
//      num : 读出的数据个数。
//注意 : 通过该方式读出子串口缓存中的数据。单次不能超过 16 个。
/*****
***/
void WkReadSFifo(u8 port,u8 *rec,u8 num)
{
    u8 n,cmd;
    cmd=0xc0|((port-1)<<4)|(num-1);
    uart_sendByte(cmd);
    for(n=0;n<num;n++)
    {
        rec[n]=uart_recByte();
        // printf("0x%x!\n",rec[n]);
    }
}
```

波特率自适应函数

```
/*****Wk_BaudAdaptive*****/
***/
//函数功能:主串口波特率匹配
/*****
void Wk_BaudAdaptive(void)
{
    /*拉低 RST 引脚 10ms, 复位 WK 芯片*/
    GPIO_ResetBits(GPIOA,GPIO_Pin_4);/*拉低*/
    delay_ms(10);
}
```

```

/*拉高 RST 引脚，完成复位，wk 进入正常工作状态*/
    GPIO_SetBits(GPIOA,GPIO_Pin_4);
    delay_ms(100);
    /*复位完成以后，发送 0x55，实现主串口波特率匹配*/
    uart_sendByte(0x55);
    delay_ms(100);
}

```

WK2114芯片的初始化

```

/*****WkInit*****/
//函数功能：初始化子串口
/*****/
void Wk_Init(u8 port)
{
    u8 gena,grst,gier,sier,scr;
    //使能子串口时钟
    gena=WkReadGReg(WK2XXX_GENA);
    gena=gena|(1<<(port-1));
    WkWriteGReg(WK2XXX_GENA,gena);
    //软件复位子串口
    grst=WkReadGReg(WK2XXX_GRST);
    grst=grst|(1<<(port-1));
    WkWriteGReg(WK2XXX_GRST,grst);
    //使能子串口总中断
    gier=WkReadGReg(WK2XXX_GIER);
    gier=gier|(1<<(port-1));
    WkWriteGReg(WK2XXX_GIER,gier);
    //使能子串口接收触点中断和超时中断
    sier=WkReadSReg(port,WK2XXX_SIER);
    sier |= WK2XXX_RFTRIG_IEN|WK2XXX_RXOUT_IEN;
    WkWriteSReg(port,WK2XXX_SIER,sier);
    //初始化 FIFO 和设置固定中断触点
    WkWriteSReg(port,WK2XXX_FCR,0xFF);
    //设置任意中断触点，如果下面的设置有效，那么上面 FCR 寄存器中断的固定中断触点
    将失效
    WkWriteSReg(port,WK2XXX_SPAGE,1);//切换到 page1
    WkWriteSReg(port,WK2XXX_RFTL,0x40);//设置接收触点为 64 个字节
    WkWriteSReg(port,WK2XXX_TFTL,0x10);//设置发送触点为 16 个字节
    WkWriteSReg(port,WK2XXX_SPAGE,0);//切换到 page0
    //使能子串口的发送和接收使能

```

```
scr=WkReadSReg(port,WK2XXX_SCR);
scr|=WK2XXX_TXEN|WK2XXX_RXEN;
WkWriteSReg(port,WK2XXX_SCR,scr);
}
```

子串口初始化程序

```
/******Wk_DeInit******/
//函数功能：初始化子串口
/*******/
void Wk_DeInit(u8 port)
{
    u8 gena,grst,gier;
    //关闭子串口总时钟
    gena=WkReadGReg(WK2XXX_GENA);
    gena=gena&(~(1<<(port-1)));
    WkWriteGReg(WK2XXX_GENA,gena);
    //使能子串口总中断
    gier=WkReadGReg(WK2XXX_GIER);
    gier=gier&(~(1<<(port-1)));
    WkWriteGReg(WK2XXX_GIER,gier);
    //软件复位子串口
    grst=WkReadGReg(WK2XXX_GRST);
    grst=grst|(1<<(port-1));
    WkWriteGReg(WK2XXX_GRST,grst);
}
```

子串口波特率设置函数

```
/******Wk_SetBaud*****
*****/
//函数功能：设置子串口波特率函数、此函数中波特率的匹配值是根据 11.0592Mhz 下的外部晶振计算的
// port:子串口号
// baud:波特率大小.波特率表示方式,
//
/******Wk2114SetBaud*****
*****/
void Wk_SetBaud(u8 port,enum WKBaud baud)
{
    unsigned char baud1,baud0,pres,scr;
```

//如下波特率相应的寄存器值，是在外部时钟为 11.0592mhz 的情况下计算所得，如果使用其他晶振，需要重新计算

关于波特率计算问题：

时钟频率/（波特率*16）：

1、整数部分减 1 后变成 16 进制写入{BAUD1,BAUA0}。

2、小数部分应该这样：（小数部分*16）取整数写入 PRES。

例如：系统 29.4912Mhz 需要的波特率是 500K

$29491200/(16*500000)=3.6864$

BAUD1,BAUA0=3-1=2; PRES=0.6864*16=10.9824=11

```
switch (baud)
```

```
{
```

```
case B600:
```

```
    baud1=0x4;
```

```
    baud0=0x7f;
```

```
    pres=0;
```

```
    break;
```

```
case B1200:
```

```
    baud1=0x2;
```

```
    baud0=0x3f;
```

```
    pres=0;
```

```
    break;
```

```
case B2400:
```

```
    baud1=0x1;
```

```
    baud0=0x1f;
```

```
    pres=0;
```

```
    break;
```

```
case B4800:
```

```
    baud1=0x00;
```

```
    baud0=0x8f;
```

```
    pres=0;
```

```
    break;
```

```
case B9600:
```

```
    baud1=0x00;
```

```
    baud0=0x47;
```

```
    pres=0;
```

```
    break;
```

```
case B19200:
```

```
    baud1=0x00;
```

```
    baud0=0x23;
```

```
pres=0;
break;
case B38400:
    baud1=0x00;
    baud0=0x11;
    pres=0;
    break;
case B76800:
    baud1=0x00;
    baud0=0x08;
    pres=0;
    break;
case B1800:
    baud1=0x01;
    baud0=0x7f;
    pres=0;
    break;
case B3600:
    baud1=0x00;
    baud0=0xbf;
    pres=0;
    break;
case B7200:
    baud1=0x00;
    baud0=0x5f;
    pres=0;
    break;
case B14400:
    baud1=0x00;
    baud0=0x2f;
    pres=0;
    break;
case B28800:
    baud1=0x00;
    baud0=0x17;
    pres=0;
    break;
case B57600:
    baud1=0x00;
    baud0=0x0b;
```



```
        pres=0;
        break;
    case B115200:
        baud1=0x00;
        baud0=0x05;
        pres=0;
        break;
    case B230400:
        baud1=0x00;
        baud0=0x02;
        pres=0;
        break;
    default:
        baud1=0x00;
        baud0=0x00;
        pres=0;
}

//关掉子串口收发使能
scr=WkReadSReg(port,WK2XXX_SCR);
WkWriteSReg(port,WK2XXX_SCR,0);
//设置波特率相关寄存器
WkWriteSReg(port,WK2XXX_SPAGE,1);//切换到 page1
WkWriteSReg(port,WK2XXX_BAUD1,baud1);
WkWriteSReg(port,WK2XXX_BAUD0,baud0);
WkWriteSReg(port,WK2XXX_PRES,pres);
WkWriteSReg(port,WK2XXX_SPAGE,0);//切换到 page0
//使能子串口收发使能
WkWriteSReg(port,WK2XXX_SCR,scr);
}
```

子串口接收发送数据函数

```
/******Wk_SendData******/
//本函数为子串口发送数据的函数，发送数据到子串口的 FIFO.然后通过再发送
//参数说明：port：子串口端口号
//          *sendbuf:需要发送的数据 buf
//          len：需要发送数据的长度
// 函数返回值：实际成功发送的数据
//说明：调用此函数只是把数据写入子串口的发送 FIFO，然后再发送。1、首先确认子串口的发送 FIFO
有多少数据，根据具体情况、
//确定写入 FIFO 数据的个数，
```

```
/******
```

```
unsigned int Wk_SendData(unsigned char port,unsigned char *sendbuf,unsigned int len)
{
```

```
    unsigned int ret,tfcnt,sendlen,num,slen;
```

```
    unsigned char  fsr;
```

```
    slen=len;
```

```
    if(slen<=0)
```

```
        {return 0;}//检查长度 是否大于 0
```

```
    fsr=WkReadSReg(port,WK2XXX_FSR);
```

```
    if(~fsr&WK2XXX_TFULL )
```

```
    {
```

```
        tfcnt=WkReadSReg(port,WK2XXX_TFCNT);
```

```
        sendlen=256-tfcnt;//计算发送 FIFO 剩余空间
```

```
        ret=sendlen<=slen?sendlen:slen;//计算实际发送数据的长度
```

```
        slen=ret;
```

```
        while(slen)//发送 slen 长度的数据。
```

```
        {
```

```
            num=slen>=16?16:slen;
```

```
            slen=slen-num;
```

```
            WkWriteSFifo(port,sendbuf,num);
```

```
            sendbuf=sendbuf+num;
```

```
        }
```

```
    }
```

```
    return ret;//返回实际发送数据长度
```

```
}
```

```
/******Wk_ReceiveData*****
```

```
//本函数为子串口接收数据函数，先确定 FIFO 中子串口个数，然后一次性读出数据
```

```
//参数说明：port：子串口端口号
```

```
//      *sendbuf:需要发送的数据 buf
```

```
//
```

```
// 函数返回值：实际成功接收到数据长度
```

```
//说明：调用此函数只是把 fifo 中数据的个数读出来
```

```
//
```

```
/******
```

```
unsigned int Wk_ReceiveData(unsigned char port,unsigned char *recbuf)
```

```
{
```

```
    unsigned char fsr,tfcnt;
```

```
    unsigned int rlen,len,re;
```

```
    fsr=WkReadSReg(port,WK2XXX_FSR);
```

```
if(fsr&WK2XXX_RDAT)
{
    tfcnt=WkReadSReg(port,WK2XXX_RFCNT);//获取接收 fifo 中数据个数
    rlen=(tfcnt==0)?256:tfcnt;
    re=rlen;
    while(rlen)
    {
        if(rlen>=16)
        {
            WkWriteSFifo(port,recbuf,16);
            recbuf=recbuf+16;
            rlen=rlen-16;
        }
        else
        {
            WkWriteSFifo(port,recbuf,rlen);
            rlen=0;
        }
    }
}
```

测试发送FIFO剩余空间长度函数

```
/******WK_TxLen******/
//函数功能:获取子串口发送 FIFO 剩余空间长度
// port:端口号
// 返回值 : 发送 FIFO 剩余空间长度
/******WK_Len******/
int wk_TxLen(u8 port)
{
    u8 fsr,tfcnt;
    int len=0;
    fsr =WkReadSReg(port,WK2XXX_FSR);
    tfcnt=WkReadSReg(port,WK2XXX_TFCNT);
    if(fsr& WK2XXX_TFULL)
    { len=0;}
    else
    {len=256-tfcnt;}

    return len;
}
```

```
}
```

子串口发送/接收函数

```
/******WK_TxChars******/
//函数功能:通过子串口发送固定长度数据
// port:端口号
// len:单次发送长度不超过 256
//
/******WK_TxChars******/
int wk_TxChars(u8 port,int len,u8 *sendbuf)
{
    #if 0
        /*读 FIFO 方式*/
        int slen;
        while(len)//发送 len 长度的数据。
        {
            slen=len>=16?16:len;
            len=len-slen;
            WkWriteSFifo(port,sendbuf,slen);//通过 fifo 方式发送数据
            sendbuf=sendbuf+slen;
        }
    #else
        /*读寄存器方式*/
        int num=len;
        for(num=0;num<len;num++)
        {
            WkWriteSReg(port,WK2XXX_FDAT,*(sendbuf+num));
        }
    #endif
}

/******WK_RxChars******/
//函数功能:读取子串口 fifo 中的数据
// port:端口号
// recbuf:接收到的数据
// 返回值:接收数据的长度
/******WK_RxChars******/
int wk_RxChars(u8 port,u8 *recbuf)
{
    u8 fsr,rfcnt;
    int n,len=0,num=0;
    fsr =WkReadSReg(port,WK2XXX_FSR);
    rfcnt=WkReadSReg(port,WK2XXX_RFCNT);
    /*判断 fifo 中数据个数*/
```

```
if(fsr& WK2XXX_RDAT) //判断是否为空, 0 为空, 1 不为空
{
    len=(rfcnt==0)?256:rfcnt;
}
num=len;
/*读取接收 fifo 中的数据*/
#if 0
/*读 FIFO 方式*/
while(len)
{
    if(len>=16)
    {
        WkReadSFifo(port,recbuf,16);
        recbuf=recbuf+16;
        len=len-16;
    }
    else
    {
        WkReadSFifo(port,recbuf,len);
        len=0;
    }
}
#else
/*读寄存器方式*/
for(n=0;n<len;n++)
    *(recbuf+n)=WkReadSReg(port,WK2XXX_FDAT);
#endif
return num;
}
```

中断处理函数

```
/******WK_IrqApp******/
//函数功能:中断处理的一般流程, 可以根据需要修改
/******WK_IrqApp******/
//u8 WK_IrqApp(void)
//{
//    u8 gifr,sier,sifr;
//    gifr=WkReadGReg(WK2XXX_GIFR);
//    if(gifr&WK2XXX_UT1INT)//判断子串口 1 是否有中断
//    {
//
```

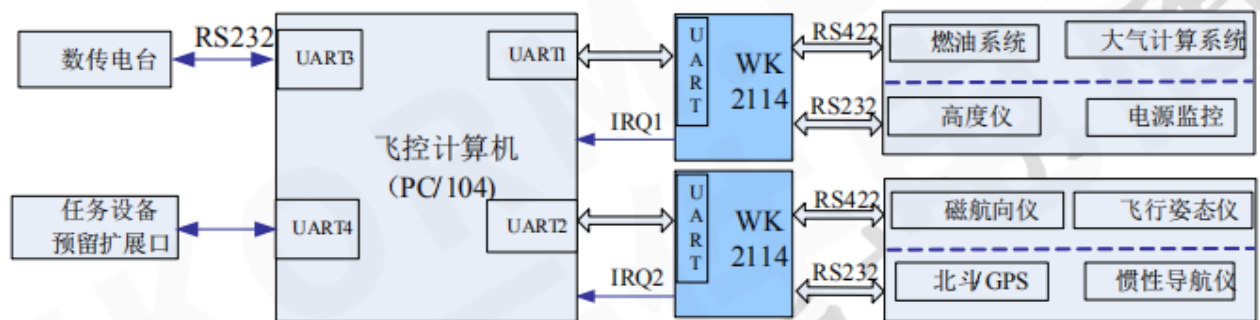

芯片，存储芯片，外围接口芯片等，再接众多的外设模块构成，可选的嵌入式操作系统有 Wince, linux, VxWorks 等，再加上导航引擎，电子地图和用户操作界面等组成。

WK2114 主接口为 UART，嵌入式平台通过 UART 接口同 WK2114 相连，WK2114 可以扩展 4 个子串口分别连接 GPS 模块，倒车雷达模块，蓝牙模块，胎压监测等。

6.2 基于 PC/104 构架无人机机载计算机串口扩展应用方案

无人机广泛采用的 PC/104 机载计算机是一种模块化，高可靠，可扩展的嵌入式工业计算机，广泛应用于工业控制、航空航天等领域。无人机系统中，部分传感器、控制模块、任务模块通常采用串行接口通信。随着无人机的发展，需要接入的外设和传感器越来越多，PC/104 计算机自带的 4 个串口在实际应用中不能完全满足需求，需要进行串口扩展。

本设计方案中选用 1 路 UART 串口扩展 4 路 UART 串口的 WK2114 进行串口扩展，原理示意图如下图。PC/104 模块的串口 1 和 2 分别连接 2 片 WK2114，扩展出 8 个硬件串口，通过 RS422/232 电平转换后与多种外部外设连接。



PC/104架构下的串口扩展串口设计方案

该应用方案优势：

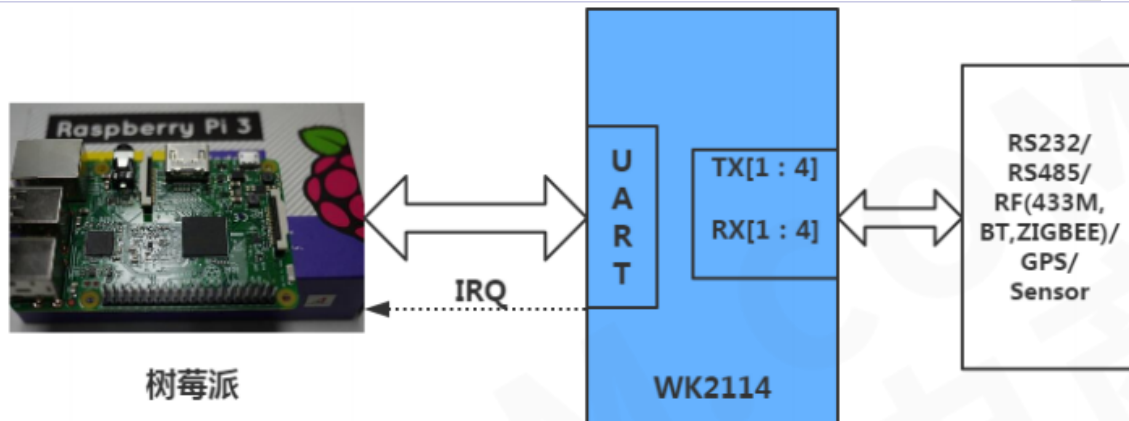
- 1、简单灵活：通过主板的 1 个串口扩展出 4 个标准硬件串口，主串口工作速率>115kbps，扩展串口工作在 9.6-38.4kbps，最多可以扩展出 16 个硬件串口，简单灵活。
- 2、稳定高效：相对于 PC/104 自带串口的 16 级 FIFO，扩展的硬件串口具备 256 级收发独立 FIFO，可以提供更大的输入输出缓存，有效提高数据收发的稳定性。
- 3、更高实时性：扩展串口通过优先级更高的外部中断与 PC/104 连接，中断触发可设置，能有效提高系统实时性。
- 4、方便测试和模拟仿真：所有外设都通过串口接入，可以在 PC/104 的串口接入串口测试和模拟仿真设备，方便的实现测试和模拟仿真。

6.3 “树莓派”系统的串口扩展应用和教学示例

树莓派是只有信用卡大小，成本仅 25-35 美元的微型计算机，可搭载开源的 Linux 系统和应用，由 Raspberry Pi 非营利基金会在 2012 年推出，2016 年推出的升级版 Pi

3, 处理器速度比 Pi 1 快 10 倍, 内存也加倍, 并且能搭载 Windows 10。到 2015 年, 全球的树莓派已经销售了千万套, 用户中教育、工业、创客用户大致各占三分之一。

树莓派的 GPIO 中自带一个 UART 串口, 该串口默认为系统调试接口, 实际应用中当树莓派需要接更多的外设如各种传感器、控制模块、GPS、蓝牙、红外模块等, 这些设备多广泛使用串行接口 (UART\RS232\485), 因此需要对树莓派进行串口扩展。本方案中选用 WK2114 进行串口扩展, 可以通过跳线选择, 灵活实现通过树莓派上的 UART 接口扩展 4 个增强型串口。



该应用方案优势:

1、 稳定运行: 现有树莓派扩展串口方案一般是通过多个 USB 转换串口实现, 在实际应用多个中该方案普遍存在工作不稳定的问题。通过 UART 接口桥接扩展串口, 是工业领域普遍采用的串口扩展方式, 具备较高的稳定性, 适用于需要长期稳定运行的系统使用。

2、 灵活扩展: WK2114 提供 UART 接口与树莓派连接。扩展出来的 4 个增强型串口, 可以连接 RS232, 蓝牙红外等各种接口, 实现多种外设的灵活接入。

3、 开源学习: 本参考设计给出了树莓派在 Linux 下的 UART 扩展串口及控制 GPIO 的驱动代码和示例, 为深入学习树莓派 Linux 的设备驱动编程提供了很好的手段。

7. FAQ 汇总

1、 WK2114 串口扩展芯片的调试步骤

1. 首先是检查硬件电路, 特别是时钟和复位。
2. 其次是确认 CPU 和 WK2114 之间的通信是否正常, 通常先由 mcu 发送 0x55 实现波特率匹配, 然后我们可以读 WK2114 的 GENA 这个寄存器来确认。在读操作时序没有问题的情况下, 然后再调试写寄存器时序。
3. 了解初始化芯片的一般操作, 参见我们提供的例程和数据手册。
4. 最后实现数据的发送和接收。

2、 WK2114 串口扩展芯片的时钟电路的重要性。

答：由于串口的波特率发生器依赖于时钟，所以如果时钟没有，那么串口是不会工作的，主串口和子串口都不能工作。

3、CPU 与 WK2114 之间通信原理

CPU 和 WK2114 之间的通信原理很简单。都是通过不同的总线去读写 WK2114 芯片的寄存器，也就是通过 CPU 的总线接口接收或者发送数据，但是需要按照 WK2114 的操作时序来进行(重要)。由于 WK2114 芯片内部有相关的协议解析单元，来识别 CPU 对它的操作。如果时序或者命令格式不对，WK2114 可能不能做出正确的应答，甚至导致整个操作时序错误。这个时候需要对芯片进行复位操作。

4、CPU 与 WK2114 无法通信

波特率匹配失败，对芯片进行复位，同时 ir 引脚上拉，重新进行自适应匹配，通过读写全局寄存器 GENA 来判断是否匹配成功。

5、子串口波特率不一致

每个子串口的波特率都需要单独配置，在写寄存器时，因为页不同，先切换 page 后，再进行写操作。